CMPT 383-D100 Quiz 1 (Python)
Fall 2026

Solutions

Duration	30 minutes for the entire exam.
Aids allowed	Pencil or pen, and eraser. No notes, no papers, no books, no computers, no calculators, no cheat sheets, etc.
During the exam	Raise your hand if you would like to speak with a proctor. To be fair to all students, we don't answer questions about the exam. Choose the answer based on your best understanding of the question. Re-reading the question may help.

1) Write a **list comprehension** that evaluates to a list of **all bit-sequences of length 4**. It should contain just the 16 different 4-bit sequences as tuples of `int`s. The order of the tuples in the list doesn't matter.
The returned list should look like `[(0, 0, 0, 0), (0, 0, 0, 1), …]`.

Solution

```
[(a, b, c, d) for a in [0, 1]
              for b in [0, 1]
              for c in [0, 1]
              for d in [0, 1]]
```

2) Write a function called `transpose(matrix)` that uses **`zip` and a list comprehension** to return the *transpose* of the given matrix. Assume the matrix is a list of lists, and each internal list is the same length. For example:

```
table = [[1, 2, 3], [4, 5, 6]]
print(transpose(table))   # prints: [[1, 4], [2, 5], [3, 6]]
```

Your function should work with *any* matrix (not just the example!), and the function body should consist of a single call to `return`.

Solution

```
def transpose(matrix):
    return [list(row) for row in zip(*matrix)]
```

Solutions

3) State what must be true for a Python object to be an **iterator**. In other words, what is Python's **iterator protocol**?

Solution

The object must have an `__iter__` method, a `__next__` method, and when the iteration is complete it must raise `StopIteration`.

4) Write a **generator** called `enumerate_rev(lst)` that takes a list `lst` and returns an iterator that iterates over the elements of `lst` in reverse order as shown:

```
lst = ["cup", "bar", "hat", "bag"]
for value in enumerate_rev(lst):
     print(value)
```

what gets printed

```
('bag', 3)
('hat', 2)
('bar', 1)
('cup', 0)
```

Be precise with the output values. In particular, be careful with the index values: we want to see them exactly correct (off by 1 or more will count as a small deduction!).

Solution

```
def enumerate_rev(lst):
    n = len(lst)
    for i in range(n):
        yield lst[n - i - 1], n - i - 1
```

Solution